Project 1: SIMD Advantage Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We quantify scalar-versus-SIMD speedup for four single-threaded numeric kernels (SAXPY, dot product, elementwise multiply and a 3-point stencil) on an Intel Core i9-10900F (Comet Lake, AVX2+FMA, 256 bit vectors) running at a fixed 2.8 GHz. With cache-resident data a `-march=native` build reaches about 5.8× over scalar for SAXPY (8 float32 lanes), while the generic auto-vectorized (SSE, 128 bit) build reaches about 3×; once the working set spills to DRAM the gain compresses to 1.7–1.9× for SAXPY (1.4–1.5× for elementwise multiply, 2.7–2.8× for the stencil; ranges span $N = 2^{24}$–2^{25} and three separate sweeps). There the native kernels sit *on* the measured DRAM bandwidth roof, and the speedup that remains only reflects that scalar SAXPY, which hardware counters show needs ≈ 1.8 cycles of core work per element, cannot keep up with memory at this clock. Float64 roughly halves the SIMD advantage (4 lanes), and non-unit stride or gather access erases it entirely; an explicit AVX2 hardware gather (`vgatherdps`) is even 1.5× *slower* than the scalar gather in cache. Our negative case is memory-bound SAXPY: the loop provably vectorizes, yet wider vectors buy nothing in DRAM because it already runs at the bandwidth roof; a second anomaly is the dot product, whose non-reassociated reduction leaves a 4-cycle scalar dependency chain and ≈ 1× speedup despite packed multiplies. A cache-aware roofline whose ceilings are measured by separate microbenchmarks—an FMA-throughput loop (89.1 GFLOP/s float32, 32.0 FLOP/cycle) and a STREAM-style triad per cache level (from 207 GiB/s in L1 down to 24.3 GiB/s from DRAM)—places the L3- and DRAM-resident native streaming kernels at 96 % to 118 % of their bandwidth roof (the triad roof is a reference; SAXPY's two-reads-one-write pattern sustains slightly more).

1 Introduction

Single-instruction-multiple-data (SIMD) units let one instruction operate on a whole vector of elements. On the Intel Core i9-10900F studied here the widest available unit is AVX2 with FMA, a 256 bit register that holds 8 float32 or 4 float64 lanes, so the *ideal* speedup over scalar code is 8× (float32) or 4× (float64). Real kernels rarely reach that ceiling: loads and stores, loop prologue/epilogue handling, reduction dependencies, and—above all—finite memory bandwidth cap the benefit. The purpose of this report is to measure where the benefit appears, by how much it falls short of the lane count, and exactly when it disappears.

This report answers:

1. How much faster are auto-vectorized builds than scalar builds for each kernel, and how does that change from cache-resident to DRAM-resident working sets? (Sections 5 and 6)

2. What do alignment, loop tails, stride and gather access, and data type cost? (Sections 7 to 9)

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	Each run is pinned to one physical core (`--cpu 2`, `sched_setaffinity`); SMT is disabled, so the core has no sibling thread, and no other pinning/isolation is used.; no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 2); FIO: fio-3.42; MLC: not available (not installed)
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
Env. snapshot	2026-10-21 19:52 EDT (each raw run directory also stores its own `env.txt`)

3. Where do these kernels sit in a roofline model of this machine? (Section 10)

4. When does the expected SIMD benefit fail to appear? (Section 11)

In brief: SIMD helps most when data fits in L1/L2, where `native` gives roughly 5.8× for SAXPY and the cache feeds the vector units fast enough to approach the lane count; the advantage shrinks step-by-step as the working set crosses L2 and L3 and falls to 1.7–1.9× for SAXPY in DRAM (1.4–1.5× multiply, 2.7–2.8× stencil), where the vector build runs at the memory-bandwidth roof and only the scalar build's own instruction cost keeps the ratio above 1×. Data type matters through lane count (float64 roughly halves the gain) and access pattern matters even more (any stride ≥ 2 removes the SIMD advantage; gather, even with AVX2 `vgatherdps`, never gains). Two cases show the limits of "it vectorized, so it must be faster": memory-bound SAXPY (Section 11) and the dot-product reduction (Section 4).

2 Methodology

2.1 Platform

This CPU (Comet Lake, client Skylake core) implements SSE/AVX/AVX2 and FMA but *not* AVX-512, so the widest usable vector register is 256 bit (`ymm`): 8 lanes of float32 or 4 of float64. With `-march=native` GCC targets AVX2+FMA and prefers 256 bit vectors; the default (generic `x86-64`) target only guarantees SSE2, so the plain auto-vectorized build emits 128 bit (`xmm`) code. That 256-vs-128-bit gap is the main reason we keep `autovec` and `native` as separate variants (Table 3).

2.2 Kernels

We count a multiply and an add as one FLOP each, so a fused multiply–add is two FLOPs; SAXPY and the dot product are therefore 2 FLOP/element (one FMA), elementwise multiply is 1, and the stencil is 5 (three multiplies, two adds). All four kernels from the project brief are implemented and

Table 2: Kernels and floating-point operations (FLOPs) per processed element.

Kernel	Definition	FLOPs / element
SAXPY	$y_i \leftarrow a\,x_i + y_i$	2
Dot product	$s \leftarrow \sum_i x_i\,y_i$	2
Elementwise multiply	$z_i \leftarrow x_i\,y_i$	1
3-point stencil	$y_i \leftarrow a\,x_{i-1} + b\,x_i + c\,x_{i+1}$	5

Table 3: Build variants. Only the kernel translation unit (`src/kernels.c`) differs; the timing harness (`main.c`, `common.c`) and the reference (`reference.c`) are compiled identically across variants.

Variant	Kernel compiler flags (from `simd_bench --info`)
scalar	`-O3 -fno-tree-vectorize -fno-tree-slp-vectorize`
autovec	`-O3` (generic `x86-64` ⇒ SSE2, 128 bit)
native	`-O3 -march=native` (AVX2+FMA, 256 bit)
native512	not available on this CPU (no AVX-512; 256 bit is the widest ISA)

validated; SAXPY is used as the representative kernel for the locality, alignment and stride studies because it has the simplest unit-stride access and a well-known arithmetic intensity.

2.3 Build variants

The primary compiler is GCC gcc (GCC) 16.2.1 20260810. The reference implementation additionally uses `-O2 -fno-tree-vectorize -ffp-contract=off` so it evaluates each expression in strict, non-fused order and stays a trustworthy oracle. We deliberately do *not* pass `-ffast-math` to any variant: this keeps float results reproducible but, as Sections 4 and 11 show, it also prevents the compiler from reassociating the dot-product reduction—an effect we wanted to measure rather than hide.

2.4 Data collection and hypotheses

All measurements in this report were collected on **2026-10-21**: every experiment in one `run.sh all` session (00:34–00:43 local time, raw directories `*/20261021-003450`), except the alignment/tail sweep. That sweep was re-run at 19:40 (`alignment/20261021-194049`) after we found that offsets 40 B to 56 B were missing from its offset list. The re-run used the same settings (governor, no turbo, no SMT, pinned to CPU 2), checked before the run. The vectorization evidence was generated the same day (Section A). Each experiment section opens with a hypothesis derived from first principles (lane counts, cache sizes, port counts). The hypotheses for the experiments added on 2026-10-21 (ceilings, extended alignment/tail, AVX2 gather, counters) were drafted before that session. The others date from an earlier, superseded turbo-clock sweep and may have been influenced by it. The repository history cannot prove either ordering. We therefore present them only as predictions that we compare against the data, and we report explicitly where the data contradicted them.

2.5 Timing and statistics

- Timer: `clock_gettime(CLOCK_MONOTONIC_RAW)` (resolution 1 ns on this host, from `simd_bench --info`). Each repetition times a calibrated batch of kernel invocations lasting at least 25 ms, i.e. roughly 2.5×10^7 timer ticks, so timer resolution and call overhead are negligible.

- Warm-up: 3 untimed invocations per configuration; repetitions: 12 per configuration.
- Reported statistic: median; error bars show the interquartile range (IQR) across the 12 repetitions. The median resists the occasional scheduler/turbo outlier better than the mean. Speedup error bars span the range implied by the two IQRs, $[\, q_{25}(\text{SIMD})/q_{75}(\text{scalar}),\ q_{75}(\text{SIMD})/q_{25}(\text{scalar}) \,]$ (`scripts/speedup_iqr.awk`), a conservative bound.
- Pinning, SMT, governor and turbo: Table 1. For the reported runs the governor is performance, turbo is disabled and SMT is off, which fixes the core at its 2.8 GHz base clock. We do not just assume the clock: the cycle counter (Section 2.6) gives a per-configuration median of 2.78 GHz to 2.79 GHz for all 96 locality and 123 ceilings configurations except one ceilings sweep point (2.47 GHz; the clock is user-mode cycles over wall time, so time spent in the kernel lowers it).
- Effect of the fixed clock: an earlier sweep (2026-10-12) ran with turbo enabled. Its dot product, which is latency-bound at exactly 0.5 FLOP/cycle (Section 10), reached 2.2 GFLOP/s, implying ≈ 4.4 GHz. Every number in this report comes from the fixed-clock runs. Compute-bound rates are $\approx 2.8/4.4$ of the turbo values; DRAM bandwidth is not, so DRAM-resident speedups over scalar are *larger* at the fixed clock (Section 6).
- Every kernel result is written through a `volatile` sink (`acs_consume_*`) and the kernels live in their own translation unit, so no loop is eliminated as dead code.
- Configuration order: the sweep is run as nested loops over size and variant (`scripts/run.sh`); it is not shuffled, which we note as a limitation.

2.6 Machine ceilings and hardware counters

The roofline (Section 10) needs a peak FLOP rate and memory bandwidths that are *not* derived from the kernels it is used to explain. A separate program, `build/ceilings/ceilings` (`src/ceilings.c`, `run.sh ceilings`), measures them:

- *Peak FLOP rate*: a register-only loop of 12 independent accumulators updated as $a \leftarrow a\,m + c$. FMA (and multiply/add) latency is 4 cycles on this core and two ports execute them, so at least $4 \times 2 = 8$ independent chains are needed to saturate both ports; 12 leaves headroom while staying in registers. Variants: scalar `mulss`+`addss` (the ceiling of our `scalar` build), scalar FMA, SSE `mulps`+`addps` on `xmm` (the ceiling of `autovec`, which has no FMA), and AVX2 `vfmadd231ps`/`pd` on `ymm` (the ceiling of `native`).
- *Bandwidth*: STREAM-style *read* ($s \mathrel{+}= a_i$), *copy* ($b_i = a_i$) and *triad* ($a_i = b_i + s\,c_i$) written with AVX2 loads/stores, at a total footprint inside each level (L1 16 KiB, L2 128 KiB, L3 8 MiB, DRAM 512 MiB) and over a fine footprint sweep from 4 KiB to 512 MiB. Bytes are counted STREAM-style (explicit loads + stores) and, separately, including the write-allocate read that a store to an unread line costs; the roofline uses the latter, *actual* traffic, and counts the kernels' bytes the same way (Table 8).

Hardware counters. We have no root. With `perf_event_paranoid` = 2 an unprivileged process may still count *user-space* events of its own threads, and doing so from inside the benchmark lets us bracket exactly the timed repetitions. `src/perfctr.c` therefore calls `perf_event_open(2)` directly for cycles, instructions, LLC misses and L1D read misses (kernel time excluded), enabled only around each timed repetition; an event that cannot be opened is reported as `NA` instead of failing.

The locality sweep records them (Section 6) and the ceilings record cycles, from which the effective clock and FLOP/cycle follow.

2.7 Data initialization and working-set sizes

Inputs are filled with a deterministic linear-congruential sequence in $[-0.5, 0.5)$ (fixed seed), which avoids denormals, NaNs and overflow; SAXPY's in-place update grows y only linearly in the repetition count, so it never overflows float32. The gather pattern uses a Fisher–Yates permutation of $[0, N)$ so every element is touched exactly once with no locality.

Each kernel streams two or three arrays, so the bytes touched are (arrays)×(element size)×N. For float32 SAXPY (two arrays, 8 B/element) the working set crosses this machine's 32 KiB L1d near $N \approx 4096$, the 256 KiB L2 near $N \approx 32\,768$, and the 20 MiB L3 near $N \approx 2.6 \times 10^6$; beyond that it is DRAM-resident. These boundaries are annotated on the locality plot (Section 6).

3 Correctness Validation

Every build variant is checked two independent ways before any of its timings are trusted (`make test`, `scripts/validate.sh`).

Reference comparison. `src/reference.c` contains plain scalar loops compiled with `-fno-tree-vectorize -ffp-contract=off`, so it evaluates each kernel in strict, non-fused, left-to-right order. The driver runs the kernel under test and the reference on identical inputs and compares element-by-element (or the scalar result, for the dot product) before timing.

Cases	kernels × {float32, float64}; N around multiples of the lane count $(1, 2, 3, 7, 8, 9, 15, 16, 17, 31, 33, 64, 127, 1000, 1024, 4095, 4096, 4097, 65536)$; start offsets 4, 8, 12, 16, 20, 28, 32 and 60 B; strides $2, 4, 8, 16$; gather; explicit AVX2 gather (native only, $N \in \{7, 8, 9, 16, 17, 1000, 4096, 65536\}$); int32 multiply
Tolerance	relative error $\leq 10^{-5}$ (float32) and $\leq 10^{-12}$ (float64). The bound is loose enough for the one legal source of disagreement—FMA contraction in the vectorized builds versus the non-fused reference—and tight enough to catch real bugs.
Command	`make test`

Known-answer self-test (`simd_bench --selftest`), independent of the reference: the dot product of two all-ones vectors equals N; $y = 2x + y$ with $x = 1, y = 3$ gives 5 everywhere; $z = x\,y$ with $x = 2, y = 3$ gives 6; and the stencil with $a = b = c = 1$ on a constant-1 input gives 3 in the interior; and the explicit AVX2 gather through a reversed permutation gives 5 everywhere. These guard against a reference and kernel sharing the same mistake.

The sweep covers sizes with and without a vector-width remainder (to exercise the scalar/masked tail) and deliberately misaligned start addresses (to exercise the loop prologue), so correctness is established precisely in the regimes the later experiments stress.

Result: all 578 validation cases pass for every variant (scalar, autovec, native); the maximum observed float32 relative error was below 10^{-6}, consistent with FMA contraction alone.

4 Vectorization Verification

Hypothesis (from first principles, then compared with the data)

The `scalar` build should emit only scalar SSE (`ss`/`sd`) instructions and report zero vectorized loops. `autovec`, limited to the generic SSE2 target, should vectorize the streaming kernels at 128 bit (4 float32 lanes). `native` should vectorize them at 256 bit (`ymm`, 8 float32 / 4 float64 lanes) and fuse the multiply–add in SAXPY and the stencil into `vfmadd`. We expect the compiler to *refuse* to vectorize the dot-product reduction usefully, because summation reassociation is illegal without `-ffast-math`.

Table 4 counts, for each kernel function and variant, the instructions that operate on vector registers or use packed operations, extracted by `objdump` in `make report`. (A few "SIMD" matches in the scalar column are register-clearing `pxor`/`movaps` on `xmm`, not vectorized loop bodies—the compiler reports *zero* vectorized loops for `scalar`.)

4.1 Compiler report

GCC's `-fopt-info-vec` confirms the widths. For SAXPY (`kernels.c:29`):

```
# native  (-O3 -march=native):
src/kernels.c:29:26: optimized: loop vectorized using 32 byte vectors and unroll factor 8
src/kernels.c:29:26: optimized: epilogue loop vectorized using 16 byte vectors and unroll factor 4
# autovec (-O3, generic x86-64):
src/kernels.c:29:26: optimized: loop vectorized using 16 byte vectors and unroll factor 4
# scalar  (-O3 -fno-tree-vectorize): 0 loops vectorized
```

32 B vectors = 256 bit = 8 float32 lanes (native); 16 B = 128 bit = 4 lanes (autovec). GCC's "unroll factor" is the vectorization factor (elements per iteration), not extra unrolling: the disassembly below has one `ymm` FMA per iteration. The 128 bit epilogue handles a remainder of 4 or more, and up to three leftover elements run as scalar iterations. The loop is *versioned* for possible aliasing between `x` and `y` (a run-time overlap check), and GCC does not peel iterations to reach an aligned start.

4.2 Disassembly

The native SAXPY inner loop is a single fused multiply–add on `ymm` registers:

```
vmovups   (%rdi,%rax,1),%ymm0           # load 8 x[i]
vfmadd213ps (%rsi,%rax,1),%ymm1,%ymm0   # ymm0 = a*x + y   (one FMA, 8 lanes)
vmovups   %ymm0,(%rsi,%rax,1)           # store 8 y[i]
```

whereas the scalar build does one element at a time with scalar SSE. It cannot fuse the multiply and add because the generic `x86-64` target it compiles for does not include the FMA instruction set (GCC's default `-ffp-contract=fast` would contract them if FMA were available, as it does in `native`):

```
movss (%rdi,%rax,4),%xmm0  # x[i]
mulss %xmm1,%xmm0          # a*x[i]
addss (%rsi,%rax,4),%xmm0  # + y[i]   (separate mul + add: generic x86-64 has no FMA)
movss %xmm0,(%rsi,%rax,4)  # y[i]
```

The dot product is the instructive exception. Native *does* multiply eight lanes at once, but the horizontal sum is a serial chain of scalar `vaddss`:

```
vmulps (%rdi,%rax,1),%ymm1,%ymm1  # 8 products at once ...
vaddss %xmm1,%xmm0,%xmm0          # ... but summed one lane at a time,
vaddss %xmm3,%xmm0,%xmm0          #     in strict source order
...
```

Table 4: SIMD instructions per kernel function and build variant.

Variant	Function	Instructions	SIMD instructions	Widest register
scalar	saxpy_f32	15	1	xmm(128)
scalar	dot_f32	19	4	xmm(128)
scalar	mul_f32	12	0	scalar
scalar	stencil_f32	34	1	xmm(128)
scalar	saxpy_f64	14	1	xmm(128)
scalar	dot_f64	19	4	xmm(128)
scalar	mul_f64	12	0	scalar
scalar	stencil_f64	31	1	xmm(128)
scalar	mul_i32	13	0	scalar
scalar	saxpy_f32_stride	31	1	xmm(128)
scalar	saxpy_f32_gather	15	1	xmm(128)
scalar	saxpy_f32_gather_avx2	14	1	xmm(128)
autovec	saxpy_f32	67	11	xmm(128)
autovec	dot_f32	58	11	xmm(128)
autovec	mul_f32	63	6	xmm(128)
autovec	stencil_f32	121	25	xmm(128)
autovec	saxpy_f64	48	7	xmm(128)
autovec	dot_f64	42	7	xmm(128)
autovec	mul_f64	48	4	xmm(128)
autovec	stencil_f64	78	16	xmm(128)
autovec	mul_i32	69	4	xmm(128)
autovec	saxpy_f32_stride	84	12	xmm(128)
autovec	saxpy_f32_gather	14	1	xmm(128)
autovec	saxpy_f32_gather_avx2	14	1	xmm(128)
native	saxpy_f32	78	11	ymm(256)
native	dot_f32	78	19	ymm(256)
native	mul_f32	79	9	ymm(256)
native	stencil_f32	110	16	ymm(256)
native	saxpy_f64	60	11	ymm(256)
native	dot_f64	57	13	ymm(256)
native	mul_f64	69	9	ymm(256)
native	stencil_f64	88	16	ymm(256)
native	mul_i32	80	9	ymm(256)
native	saxpy_f32_stride	93	12	ymm(256)
native	saxpy_f32_gather	14	1	xmm(128)
native	saxpy_f32_gather_avx2	95	18	ymm(256)

4.3 Stride and gather code

Table 4 credits `saxpy_f32_stride` with `ymm` instructions in `native`, but that is loop *versioning*, not strided SIMD: GCC tests `s == 1` at run time and only that copy of the loop is vectorized (the same `vmovups`/`vfmadd213ps` body as SAXPY). For $s > 1$ the code is a scalar `vfmadd213ss` loop in `native` and `mulss`/`addss` in the other builds. The plain gather loop (`saxpy_f32_gather`) is not vectorized by any build ("no vectype for stmt"), so we added an explicit AVX2 version, `saxpy_f32_gather_avx2`, compiled only into `native`:

```
vmovdqu    (%rax),%ymm1                # 8 indices idx[i..i+7]
vgatherdps %ymm5,(%rdi,%ymm1,4),%ymm0  # gather 8 x[idx]
vgatherdps %ymm7,(%rsi,%ymm1,4),%ymm3  # gather 8 y[idx]
vfmadd132ps %ymm4,%ymm3,%ymm0          # one 8-lane FMA
vmovss     %xmm0,(%rsi,%rdx,4)         # ... then 8 scalar stores:
vextractps $0x1,%xmm0,(%rsi,%rdx,4)    #     AVX2 has no scatter
```

It is what a hand-vectorized gather looks like on this ISA: the loads are SIMD, the stores cannot be.

4.4 Interpretation

The evidence matches the hypothesis. `native` uses `ymm` (256 bit) with FMA for SAXPY, multiply, and the stencil; `autovec` uses `xmm` (128 bit); `scalar` is genuinely scalar. Tails are handled by a narrower vectorized epilogue (a 128 bit loop), not a scalar remainder—there are no AVX-512 mask registers here because the CPU has no AVX-512. The dot product shows why "the loop vectorized" is not the same as "the loop is fast": without `-ffast-math` the compiler keeps the reduction's addition order, so the packed multiplies feed a latency-bound scalar accumulator. We quantify the throughput consequence in Sections 5 and 11.

5 Scalar vs. SIMD Baseline

Hypothesis (from first principles, then compared with the data)

For compute-friendly, unit-stride kernels (SAXPY, elementwise multiply, stencil) we expect the `native` speedup to approach the float32 lane count (8×) while data is L1/L2-resident, and to fall toward 1× once the working set is DRAM-resident and all variants share the same bandwidth. `autovec` (4 lanes) should track at roughly half the `native` speedup. The dot product should barely speed up at any size because its reduction does not reassociate (Section 4).

Fixed	float32; unit stride; aligned arrays; one thread pinned to CPU 2
Varied	kernel; variant (scalar, autovec, native); $N = 2^8 \dots 2^{25}$
Repetitions	12 per point; median, IQR error bars
Script	`SIMD_Advantage_Profiling/scripts/run.sh baseline`

5.1 Analysis

At $N = 4096$ (L1-resident) the `native` speedups are SAXPY 5.75× (20.1 GFLOP/s), stencil 3.49× (17.3 GFLOP/s), elementwise multiply 3.51× and dot 1.01×. `autovec` reaches about half of `native` for SAXPY (3.01×) and 70 % to 80 % of it for multiply and stencil (2.55×, 2.80×). None reaches the full 8× lane count even in cache: each vector FMA still needs two 256 bit loads and a store, and native SAXPY needs 0.22 to 0.28 cycles per element in L1 (Section 6), against the 0.125 cycles/element (one 8-lane iteration per cycle) that two load ports and one store port would allow. So the loop is limited by load/store issue well before it is FLOP-bound.

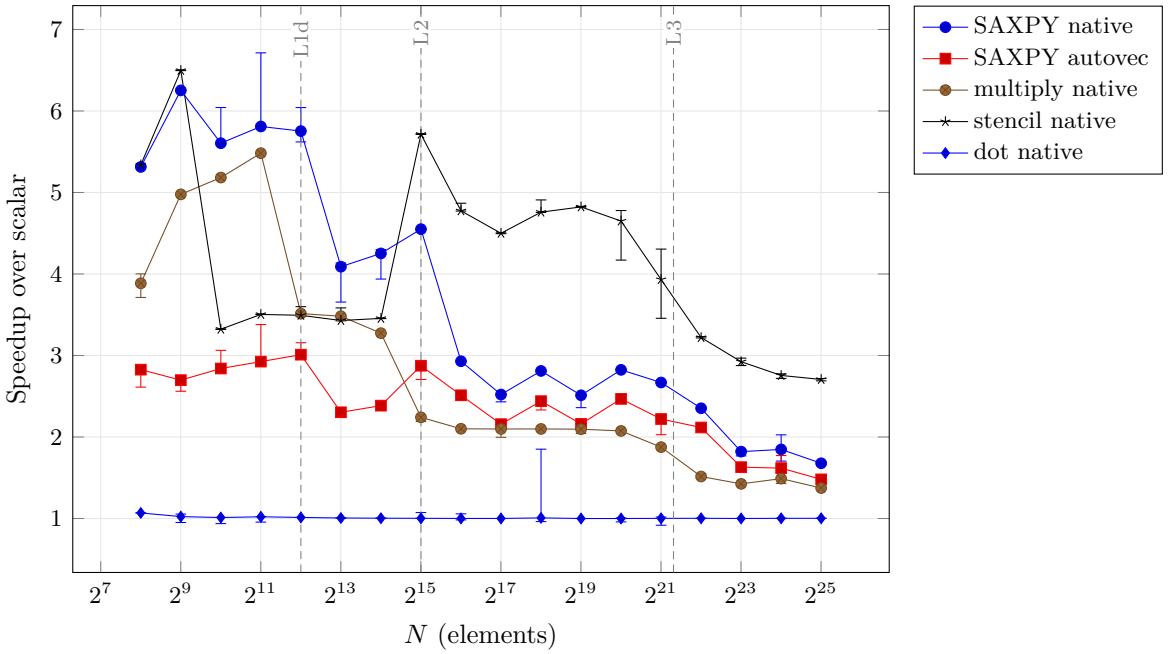


Figure 1: Speedup of the SIMD builds over scalar (median; error bars: range implied by the two IQRs, Section 2). Gains peak while data is cache-resident and compress in DRAM (at $N = 2^{24}$: SAXPY 1.85×, multiply 1.49×, stencil 2.76×); the dot product never benefits.

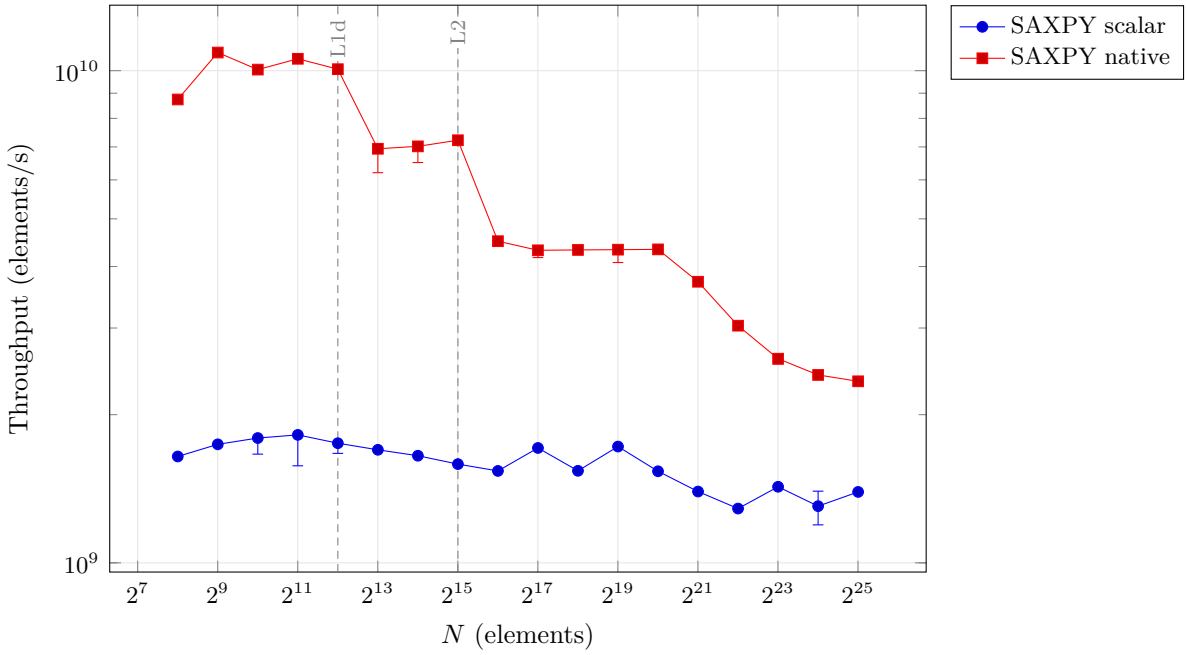


Figure 2: Achieved SAXPY throughput (elements/s, median and IQR). Native drops to a DRAM-limited plateau for large N, about 1.7–1.9× above scalar, which loses comparatively little outside cache.

The kernels mostly behave as predicted, and the exceptions are informative. *Elementwise multiply* streams three arrays (12 B/element), so it leaves L1 earlier (near $N \approx 2700$) than SAXPY. Its speedup peaks at 5.48× at $N = 2048$ and has already dropped to 3.51× at $N = 4096$. The *stencil* has the highest FLOP count (5) and reuses x_{i-1}, x_i, x_{i+1}, so it stays the most compute-bound and keeps a 2.7× advantage even in DRAM (Figure 1). Its native speedup is irregular, though: 3.3–3.5× for $N = 1024 \ldots 16\,384$ but 4.5–5.7× for the L2/L3-resident sizes. In other words, the cache-resident stencil runs *slower* per element than the L3-resident one (Section 12). The *dot product* is the reduction case from Section 4: packed multiplies, serial sum, ≈ 1× at every size, at 1.33 GFLOP/s to 1.52 GFLOP/s.

By $N = 16\,777\,216$ (128 MiB, far past L3) every speedup has compressed: SAXPY 1.85×, multiply 1.49×, stencil 2.76×, dot 1.00×. Figure 2 shows the asymmetry behind this. Native SAXPY falls from 20.1 to 4.8 GFLOP/s on a DRAM plateau, while scalar only falls from about 3.5 GFLOP/s (3.3 to 3.6 in cache) to about 2.7 GFLOP/s (2.6 to 2.8 in DRAM). The two plateaus do not merge because scalar SAXPY is close to its own instruction-throughput limit even in DRAM: it needs 1.5 to 1.8 cycles of core work per element with all data in cache and only ≈ 2.0 cycles per element from DRAM (Section 6). At the fixed 2.8 GHz clock this persistent ratio is larger than in the earlier turbo-clock sweep (≈ 1.4×), as expected if scalar is core-bound and native memory-bound. Three separate measurements of SAXPY at $N = 16\,777\,216$ (baseline, locality and negative-case sweeps) give 1.85×, 1.73× and 1.87×; this run-to-run spread is larger than the within-run IQR. The curves bend downward at the L1 ($N \approx 4096$) and L2 ($N \approx 32\,768$) boundaries; Section 6 resolves those transitions and Section 10 ties each plateau to a measured bandwidth roof.

6 Locality Sweep

Hypothesis (from first principles, then compared with the data)

For float32 SAXPY (two arrays, 8 B/element) the per-element time should be flat and low while the working set fits in L1 ($2N \cdot 4\,\text{B} \leq 32\,\text{KiB}$, i.e. $N \lesssim 4096$), step up at the L2 boundary ($N \approx 32\,768$) and again past L3 ($N \approx 2.6 \times 10^6$), and saturate in DRAM. The scalar and SIMD curves should be far apart in L1 and converge once DRAM bandwidth, which both share, dominates.

Fixed	kernel SAXPY; float32; unit stride; aligned; pinned to CPU 2
Varied	N (dense grid, powers of two and midpoints, across L1/L2/LLC/DRAM); variant
Counters	user-space cycles, instructions, LLC misses and L1D read misses per repetition via `perf_event_open` (Section 2.6); written to `locality_counters.csv` when the counters can be opened
Bandwidth	effective SAXPY bandwidth = elements/s × 12 B (read x, read and write y), compared with the independently measured triad roof of each level
Script	`SIMD_Advantage_Profiling/scripts/run.sh locality`

6.1 Analysis

The measured transitions line up with the cache sizes. While $N \lesssim 4096$ (L1-resident) native SAXPY takes 0.080 ns to 0.100 ns per element, a 5.6–6.9× speedup over scalar's 0.55 ns to 0.57 ns. Past the 32 KiB L1, at $N = 6144$, the native time jumps to 0.145 ns and the speedup drops to 4.2×; it stays at 3.9–4.2× through L2. Past the 256 KiB L2, at $N = 49\,152$, it falls to 2.8× (0.21 ns), and the L3 plateau holds 2.5–2.9× (0.23 ns). Past L3 the gap narrows but does *not* close, contradicting the

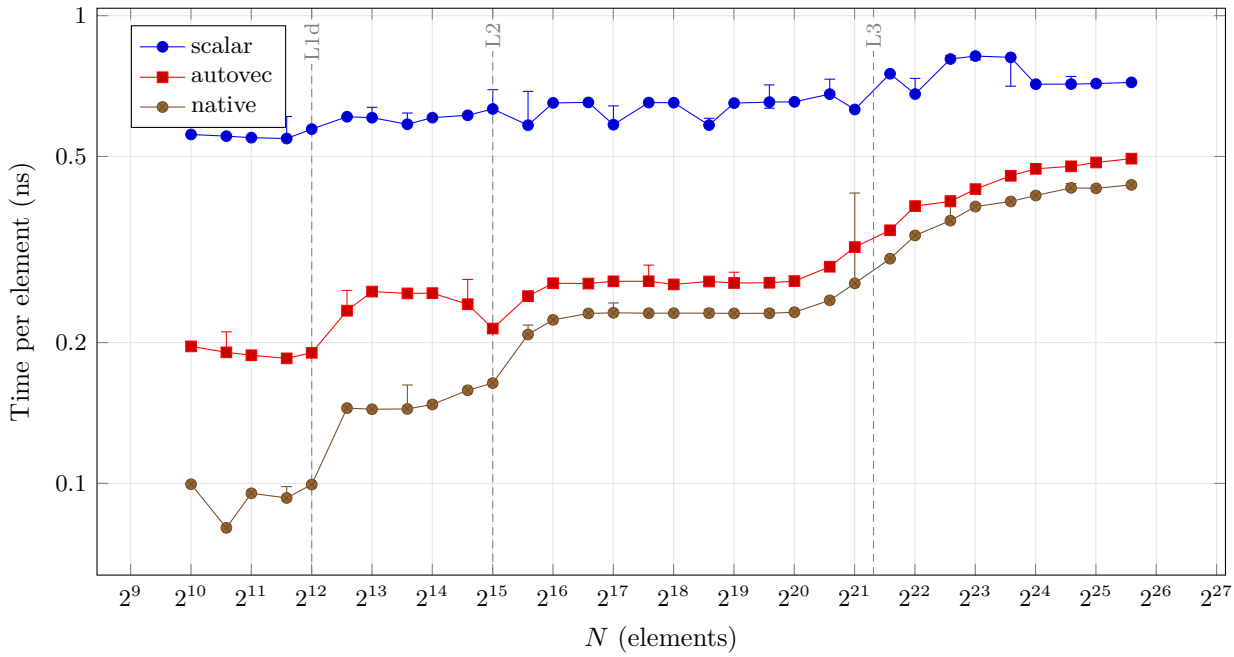


Figure 3: SAXPY time per element across cache levels. The native curve rises in steps at each cache boundary; in DRAM it stays $\approx 1.7\times$ below the scalar curve, which barely moves.

Table 5: Effective SAXPY bandwidth (GiB/s, 12 B/element) at one N inside each level, next to the triad roof measured for that level by the independent bandwidth benchmark (actual traffic incl. write-allocate, Section 2.6).

Level	N	Footprint	scalar	autovec	native	triad roof
L1	2048	16 KiB	20.4	59.5	117.3	207.0
L2	16384	128 KiB	18.5	43.8	75.8	99.4
L3	524288	4 MiB	17.2	41.7	48.4	48.3
DRAM	33554432	256 MiB	15.6	23.0	26.1	24.3

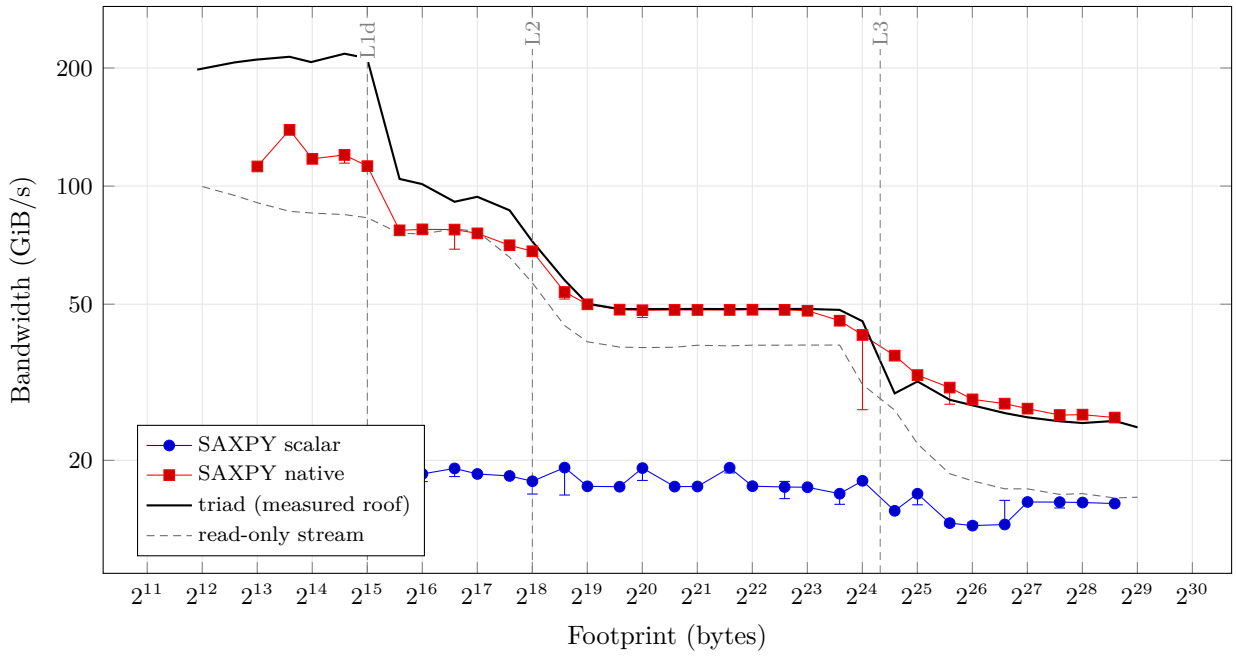
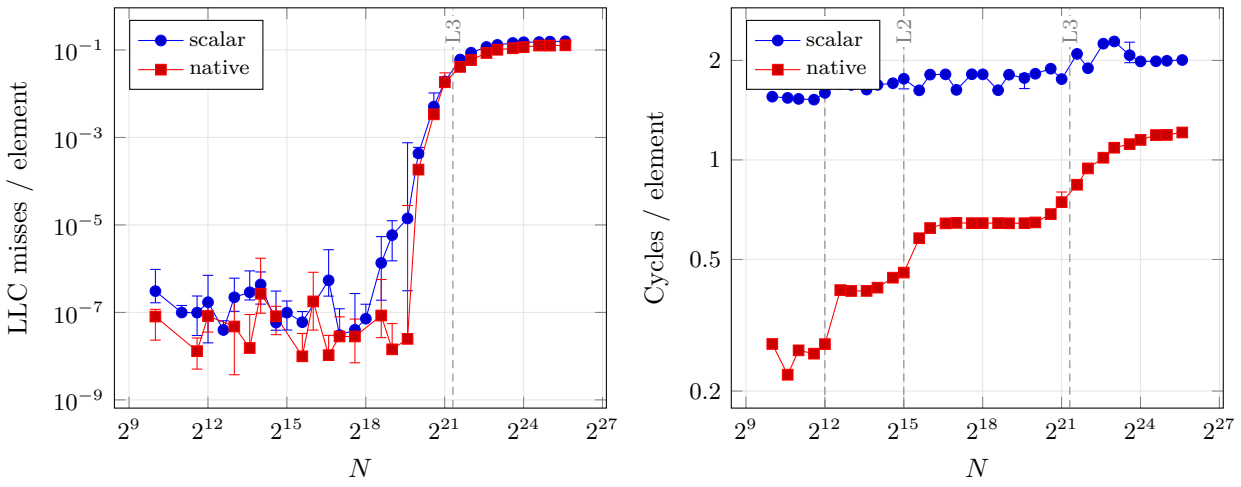


Figure 4: Effective bandwidth versus footprint: SAXPY (from Figure 3) against the independently measured triad and read-only streams. Where the native SAXPY curve meets the triad curve, SIMD SAXPY is bandwidth-bound at that level.



(a) LLC misses per element.

(b) Core cycles per element.

Figure 5: User-space hardware counters for the locality sweep (`perf_event_open`).

hypothesis: from $N = 16\,777\,216$ on, native settles at 0.41 ns to 0.43 ns per element and scalar at 0.72 ns, a remaining speedup of 1.66–1.73×.

Table 5 and Figure 4 convert the same data to bandwidth. Native SAXPY moves 117 GiB/s in L1, 75.8 in L2, 48.4 in L3 and 26.1 GiB/s from DRAM (at the representative N of the table), and each level boundary is a step in Figure 3. The independently measured triad roofs are 207, 99.4, 48.3 and 24.3 GiB/s, so native SAXPY reaches 57, 76, 100 and 107 % of the roof. It is therefore bandwidth-bound in L3 and DRAM, where it runs exactly at the measured roof. In DRAM it even slightly exceeds the triad roof, matching instead the copy stream's 26.3 GiB/s, which has the same two-reads-one-write pattern. In L1 and L2 it falls short: there the limit is the core's load/store issue rate (below), not the cache. Scalar SAXPY, by contrast, is flat at 17 GiB/s to 20.5 GiB/s through L1, L2 and L3, far below every cache roof, and drops only to 15.6 GiB/s in DRAM.

Counters (Figure 5; effective clock 2.78 GHz to 2.79 GHz at every point). LLC misses per element are below 10^{-5} while the working set fits in L3. For DRAM-resident native SAXPY they reach 0.126 ($N = 33\,554\,432$), matching the compulsory $2/16 = 0.125$ (two 64 B lines, x and y, per 16 elements; the dirty y line is written back, not missed again). So the vector loop moves no more data than it must. Scalar shows 0.155, 23 % more. We cannot attribute this with user-space counters alone; one possibility is that the generic LLC-miss event also counts L2 hardware-prefetch requests, which behave differently for a slower stream. Either way, scalar does not lose because the vector loop saves misses. The cycle counts explain the remaining gap:

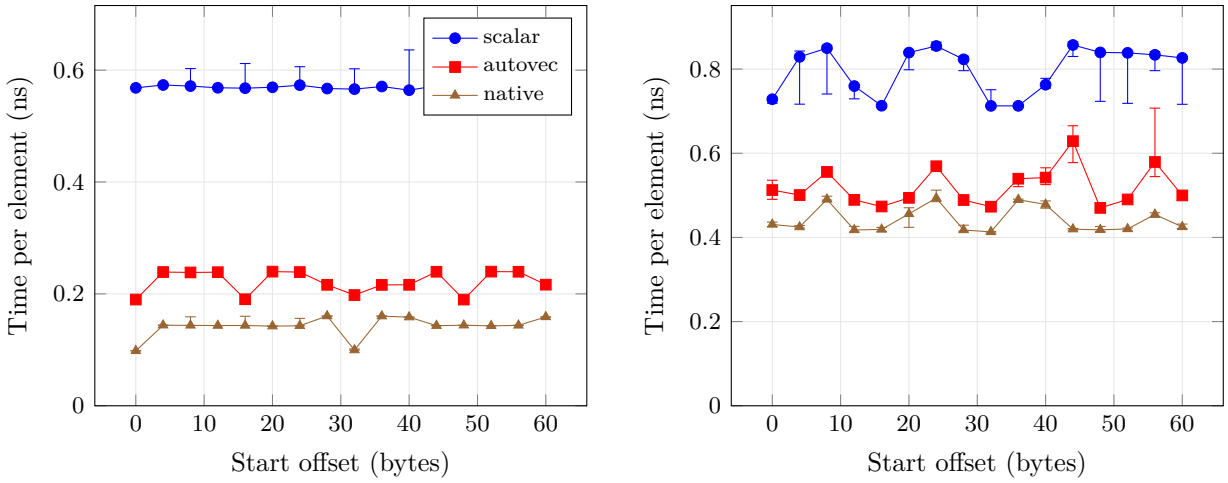
cycles/element (IPC)	L1 ($N = 2048$)	L2 ($N = 16\,384$)	L3 ($N = 524\,288$)	DRAM ($N = 33\,554\,432$)
scalar	1.53 (4.6)	1.69 (4.2)	1.81 (3.9)	1.99 (3.5)
native	0.27 (3.1)	0.41 (1.8)	0.64 (1.2)	1.19 (0.6)

Native SAXPY's cycles per element grow 4.5× from L1 to DRAM and its IPC falls to 0.6: it is waiting on memory. Scalar SAXPY already spends 1.53 cycles per element of pure core work in L1, at an IPC of 4.6. That is at this core's issue limit (4 fused micro-ops per cycle, with compare-and-branch pairs macro-fused). In DRAM it needs only 30 % more (1.99 cycles) while keeping an IPC of 3.5. So the memory system can feed scalar SAXPY almost as fast as it can execute it, and $1.99/1.19 = 1.67$ is exactly the measured DRAM speedup. This is the mechanism of the "compression" the rubric asks about. SIMD does not get slower in DRAM; once native SAXPY sits on the bandwidth roof, its advantage shrinks to the scalar loop's instruction cost divided by the memory time. That ratio depends on the clock. At the earlier turbo clock (≈ 4.4 GHz) the same scalar instruction work took less time while the memory time stayed fixed, and that sweep measured only 1.32–1.44×. Absolute native time grows step by step while scalar time is nearly flat, which is why the *ratio* shrinks at every level boundary.

7 Alignment and Tail Handling

Hypothesis (from first principles, then compared with the data)

A start address that is not 32 B-aligned should force the 256 bit loads to be unaligned and/or trigger a scalar peel prologue, costing a modest fixed overhead that is most visible at small N. Because this is a client Skylake core, unaligned `vmovups` that happen to stay within a cache line should be nearly free, so we expect the penalty to be small (tens of percent) and to recur with 64 B period as the offset sweeps a cache line.



(a) $N = 4096$ (L1-resident). (b) $N = 16\,777\,216$ (DRAM-resident).

Figure 6: Cost of a misaligned start for SAXPY, by build variant (median, IQR error bars).

Hypothesis (from first principles, then compared with the data)

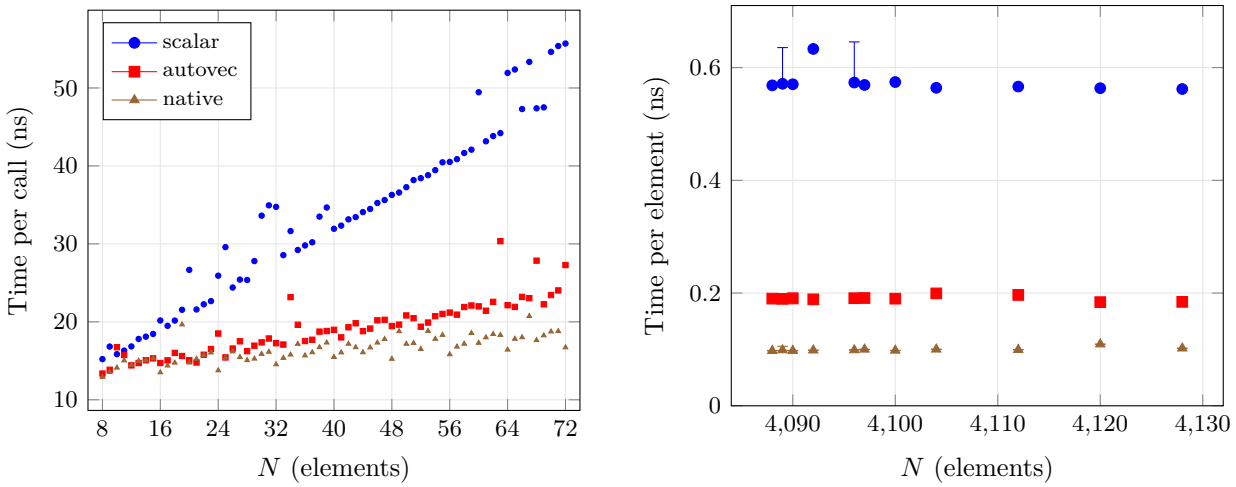
(For the extended sweep.) The `scalar` build issues 4 B loads that never straddle a line, so it should be insensitive to the offset; `autovec` (16 B loads) should show a smaller penalty than `native`. With a DRAM-resident array the misalignment cost should disappear into the memory stall time for all builds. For short loops, the time per *call* should grow in steps of 8 elements for `native` and 4 for `autovec` (one vector iteration per step), with the vectorized epilogue making the remainder cheap but not free.

Fixed	kernel SAXPY; float32; unit stride
Varied	start offsets $0, 4, 8, \ldots, 60$ B (every multiple of 4) at $N = 4096$ (L1-resident, isolates the prologue from bandwidth) and $N = 16\,777\,216$ (128 MiB, DRAM-resident); tail: every N from 8 to 72 and N stepping across an 8-lane boundary near 4096, offset 0; all three build variants
Script	`SIMD_Advantage_Profiling/scripts/run.sh alignment`

7.1 Analysis

Misaligned starts, L1-resident (Figure 6a). The penalty is real, it scales with vector width, and it repeats with the load width. Native SAXPY runs at 0.098 ns to 0.100 ns per element at offsets 0 and 32 B (both 32 B-aligned, since the base is 64 B-aligned). Every other offset costs 0.142 ns to 0.144 ns (46 %) or, at 28, 36, 40 and 60, up to 0.160 ns (63 %). `autovec` (16 B loads) is fast exactly at the multiples of 16 (offsets 0, 16, 32, 48: 0.190 ns to 0.198 ns) and 13 % to 26 % slower elsewhere. `scalar` varies by only 1.6 % (0.564 ns to 0.573 ns). This ordering, native 63 % > autovec 26 % > scalar 2 %, is what cache-line splits predict. At a non-multiple-of-32 offset every second 32 B load straddles a 64 B line, but only every fourth 16 B load does, and a 4 B scalar load never does. GCC does not peel the loop to an aligned start (Section 4), so every split load pays.

DRAM-resident (Figure 6b). At $N = 16\,777\,216$ no offset pattern survives. The spread across offsets is 19 % (native), 34 % (autovec) and 20 % (scalar), but the slow offsets do not line up with split loads. For native, offsets 8, 24, 36 and 40 are slow while 4 and 44–56 are fast, and scalar, whose loads are never split, is just as scattered. Each offset is a separate process with freshly allocated 128 MiB arrays, so this is process-to-process variation of DRAM-bound throughput. An earlier pass



(a) Short loops: time per call, $N = 8 \ldots 72$.

(b) Across an 8-lane boundary near $N = 4096$.

Figure 7: Tail handling: remainder elements processed by the vectorized epilogue (offset 0).

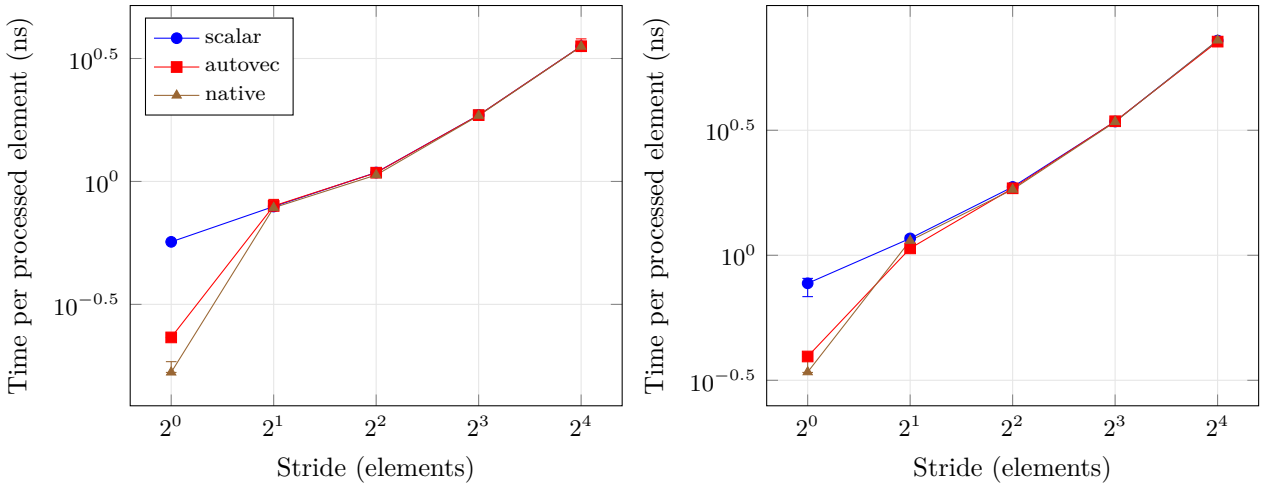
of the same sweep (00:34 session, offsets 40 to 56 still missing from the list) gave native and autovec spreads of only 2.0 % and 5.5 %, with no split-load pattern either. For the vector builds, then, the split-load cost visible in L1 disappears under DRAM misses, and alignment is a cache-resident concern only.

Tails (Figure 7). Near $N = 4096$ the time per element is flat across the 8-lane boundary (native 0.098 ns to 0.109 ns, autovec 0.184 ns to 0.199 ns, scalar 0.56 ns to 0.63 ns). With 1 to 7 leftover elements out of 4096, the remainder is invisible. Short loops expose it. For $N = 8 \ldots 72$, native's time per call is lowest at multiples of 8 (13.5 ns at $N = 16$ rising to 16.7 ns at $N = 72$, only ≈ 0.06 ns per element) and highest at $N \equiv 7 \pmod 8$, about 2 ns to 3 ns more. That sawtooth is the remainder path from the disassembly (Section 4): a 128 bit epilogue for 4 elements and then up to three scalar iterations. `autovec` grows at ≈ 0.16 ns per element and `scalar` linearly at ≈ 0.63 ns per element, both with occasional single-point outliers. Every call also pays a fixed ≈ 10 ns to 13 ns (call, loop setup and the run-time aliasing check of the versioned loop) before the first element. So the per-element speedup of short loops is far below the 5.8× at $N = 4096$: native beats scalar by only 1.18× at $N = 8$ and by 3.34× at $N = 72$. There are no AVX-512 mask registers here (no AVX-512 on this CPU); a masked tail would be the AVX-512 alternative to this narrowing epilogue.

8 Stride and Gather Access

Hypothesis (from first principles, then compared with the data)

Unit stride lets each 64 B cache line serve 16 consecutive float32 elements, so SIMD helps. As the stride grows, each vector still needs 8 elements but they come from further apart: once the stride reaches 16 elements (64 B) every element sits in its own cache line and effective bandwidth drops by up to 16×. We therefore expect time per processed element to rise roughly linearly with stride until it saturates at one cache line per element, and we expect the SIMD advantage to vanish for any non-unit stride, because the compiler must gather lanes individually. Gather (random permutation) should be the worst case.



(a) $N = 32\,768$ (cache-resident span). (b) $N = 4\,194\,304$ (DRAM-resident).

Figure 8: Effect of stride on SAXPY time per *processed* element (median, IQR error bars). The three builds coincide for every non-unit stride.

Hypothesis (from first principles, then compared with the data)

(For the extended sweep.) The explicit AVX2 gather (`vgatherdps`, Section 4) loads eight lanes per instruction but still needs one cache line per element and eight scalar stores, so we expect it to be at best modestly faster than the scalar gather in cache and no faster in DRAM. On this CPU the "Gather Data Sampling" microcode mitigation is active (`/sys/devices/system/cpu/vulnerabilities/gather_data_sampling`: "Mitigation: Microcode"), which is reported to slow `vgatherdps` considerably, so it may even lose to the scalar loop.

Fixed	SAXPY; float32
Varied	access (unit, stride 1/2/4/8/16, gather, explicit AVX2 gather); variant (scalar, autovec, native; AVX2 gather native only); $N = 32\,768$ processed elements (span 256 KiB $\times$ s: L2 for $s = 1$, L3 for $s = 16$) and $N = 4\,194\,304$ (DRAM for every s)
Definition	stride s processes elements $0, s, 2s, \ldots$ over an array of Ns elements; gather applies SAXPY through a random permutation index so every element is touched once with no spatial locality.
Script	`SIMD_Advantage_Profiling/scripts/run.sh stride`

8.1 Analysis

Table 6 makes the pattern stark. Only unit stride lets `native` beat `scalar`: 3.4–3.9$\times$ at $N = 32\,768$ and 1.99$\times$ in DRAM. The range at $N = 32\,768$ comes from the scalar build, not from the kernels. The table's *unit* row (plain `saxpy_f32`, scalar 0.635 ns, 3.90$\times$) and the stride-1 point of Figure 8a (`saxpy_f32_stride` with $s = 1$, scalar 0.568 ns, 3.39$\times$) compile to the identical scalar loop (same four instructions, same alignment, checked in the disassembly) and use the same allocation. Scalar SAXPY at this size runs in one of two states, ≈ 1.6 or ≈ 1.8 cycles per element, and the state is picked per process and occasionally changes within one. Twelve alternating check runs (six each, cycle counter on) gave 0.58 ns to 0.64 ns for *unit* and 0.57 ns to 0.65 ns for stride 1. We did not identify the cause (Section 12). At stride 2 and beyond the three builds agree within 1.00 to 1.02$\times$ (native vs. scalar) at both sizes; `autovec` is within 10 % of them everywhere, as expected since its

Table 6: Time per processed element (ns) by access pattern. *AVX2 gather* is the explicit `vgatherdps` kernel (native build only); speedup is scalar over the faster native kernel (the AVX2 gather has no strided form, hence –).

N	Access	scalar	autovec	native	AVX2 gather	speedup (×)
32768	unit	0.635	0.223	0.163	–	3.90
32768	stride 2	0.791	0.795	0.782	–	1.01
32768	stride 4	1.087	1.085	1.062	–	1.02
32768	stride 8	1.861	1.861	1.853	–	1.00
32768	stride 16	3.547	3.548	3.538	–	1.00
32768	gather	2.083	2.023	1.923	3.200	1.08
4194304	unit	0.688	0.383	0.346	–	1.99
4194304	stride 2	1.168	1.066	1.144	–	1.02
4194304	stride 4	1.876	1.853	1.833	–	1.02
4194304	stride 8	3.428	3.436	3.417	–	1.00
4194304	stride 16	7.223	7.154	7.238	–	1.00
4194304	gather	10.262	11.201	10.106	10.557	1.02

strided loop is equally scalar. Two things happen at once. First, the effective bandwidth collapses. In DRAM the time per processed element grows from stride 2 to 4 to 8 to 16 as $1.14 \to 1.83 \to 3.42 \to 7.24$ ns (native), approaching a doubling per step. Once the stride reaches 16 elements, each access pulls a fresh 64 B line and 15/16 of every line is wasted. With $N = 32\,768$ the span stays in L2/L3 and the progression is milder ($0.78 \to 1.06 \to 1.85 \to 3.54$ ns), but the near-doubling from stride 8 to 16 is the same: the cost per element is set by the cache lines touched, not by arithmetic. Second, the SIMD advantage disappears. The vectorized body exists only for $s = 1$ (GCC's run-time loop versioning, Section 4); every $s > 1$ runs the scalar copy of the loop, so no build has vector lanes to offer. Gather is the worst pattern: 10.1 ns per element in DRAM (native), 29× slower than unit stride, because the random permutation also defeats the hardware prefetcher. Even at $N = 32\,768$ it costs 1.92 ns, 12× unit stride.

Hardware gather. The explicit AVX2 kernel takes 3.20 and 10.56 ns per element at $N = 32\,768$ and $4\,194\,304$. That is 1.54× and 1.03× the time of the scalar build's gather, i.e. *slower*, decisively so when the data is cache-resident. The hypothesis above anticipated this. Each 8-lane step issues two `vgatherdps`, which the active Gather Data Sampling microcode mitigation is known to make expensive, followed by eight scalar stores. In DRAM the random line fetches dominate either way and the difference shrinks to 3 %. We could not run without the mitigation (that needs a reboot with root), so the attribution remains a hypothesis (Section 12). Either way, a SIMD gather does not recover the unit-stride advantage. The practical lesson is that access pattern, not vector width, is the first-order knob for these kernels.

9 Data Type Comparison

Hypothesis (from first principles, then compared with the data)

A 256 bit register holds 8 float32 but only 4 float64 lanes, so float64 should show roughly half the SIMD speedup of float32 for the same kernel while data is cache-resident. int32 elementwise multiply should behave like float32 (8 lanes, one `vpmulld`). In DRAM, float64 moves twice the bytes per element, so its *absolute* throughput should be lower and its speedup should compress

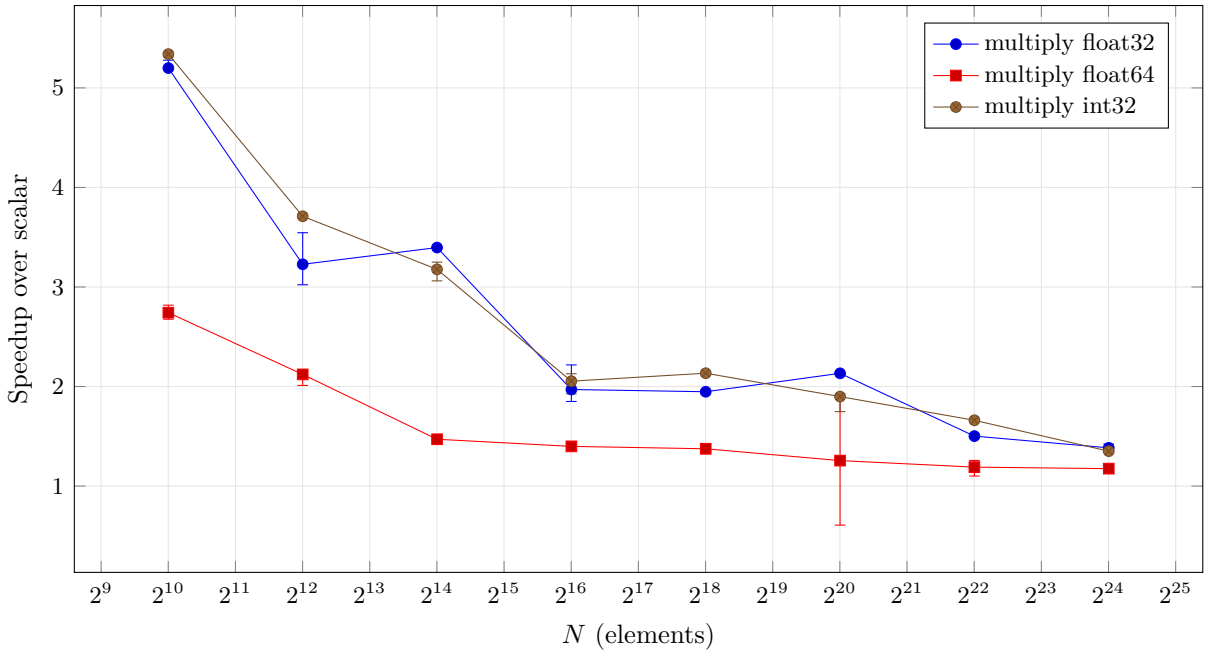


Figure 9: Native speedup of elementwise multiply by data type (median; error bars: range implied by the two IQRs). float32 and int32 (8 lanes) track each other; float64 (4 lanes) is about half.

toward 1× just as float32 does.

Fixed	unit stride; aligned
Varied	data type (float32, float64, int32); kernel; variant; $N = 2^{10} \dots 2^{24}$
Script	`SIMD_Advantage_Profiling/scripts/run.sh dtype`

9.1 Analysis

The lane-count prediction holds closely for the cache-resident multiply. At $N = 1024$ the native speedups are float32 5.20×, int32 5.34×, and float64 2.74×—float64 is 0.53× of float32, within a few percent of the expected 1/2 from halving the lanes. The float32/int32 pairing confirms the effect is lane count and not a floating-point quirk: int32 uses `vpmulld` over 8 lanes and lands right on the float32 curve. The ratio holds mid-cache too ($N = 16\,384$: f32 3.40× vs f64 1.47×), although by $N = 65\,536$, where both are L2/L3-bound, it narrows (1.97× vs 1.40×).

In DRAM the picture changes as predicted: at $N = 16\,777\,216$ every speedup has compressed (f32 1.38×, f64 1.17×). The float32 multiply figure is lower than the 1.49× the baseline sweep measured at the same N (Section 5). The two are separate runs: the native throughput agrees (1.65 vs 1.64 GFLOP/s), but the scalar run was faster here (1.19 vs 1.10 GFLOP/s). That is the same per-process scalar variability seen elsewhere (Section 12), so we quote multiply's DRAM speedup as 1.4–1.5×. Float64's *absolute* throughput is half of float32's (0.82 vs 1.65 GFLOP/s) because each element moves 8 B instead of 4 and the kernel is bandwidth-bound. So data type acts through two separate mechanisms: lane count sets the cache-resident speedup ceiling, while bytes-per-element sets the DRAM throughput. A kernel that looks "half as fast in float64" in cache is really just running 4 lanes instead of 8; the same kernel in DRAM is half as fast for the different reason that it has twice the traffic.

Table 7: Measured roofline ceilings, single core (`run.sh ceilings`). Top: register-only accumulator loops (12 independent chains); FLOP/cycle and clock from the user-space cycle counter. Bottom: STREAM-style bandwidth at one footprint per level; "incl. WA" adds the write-allocate read and is the roof used in Figure 10.

Peak loop	GFLOP/s	FLOP/cycle	GHz
scalar_f32	5.5	1.99	2.79
scalar_fma_f32	11.1	4.00	2.79
sse_f32	22.3	8.00	2.78
sse_f64	11.1	3.98	2.79
avx2_fma_f32	89.1	31.99	2.79
avx2_fma_f64	44.6	16.00	2.79

		GiB/s			
Level	Footprint (B)	read	copy	triad	triad incl. WA
L1	16,128	85.4	142.8	155.2	207.0
L2	130,944	66.2	67.4	74.5	99.4
L3	8,388,480	39.3	33.4	36.2	48.3
DRAM	536,870,784	16.0	17.6	18.3	24.3

10 Roofline Interpretation

Hypothesis (from first principles, then compared with the data)

All four kernels have very low arithmetic intensity (well under 1 FLOP/B), so on a machine whose single-core bandwidth is tens of GB/s and whose peak vector rate is over 100 GFLOP/s they should all sit on the *bandwidth* slope of the roofline when DRAM-resident, i.e. be memory-bound. The stencil, with the highest intensity, should come closest to the compute roof.

Hypothesis (from first principles, then compared with the data)

(For the measured ceilings.) Even the L1 bandwidth roof should put the ridge point above 0.5 FLOP/B, so the cache-resident native kernels should also be bounded by bandwidth (or by the load/store ports that define it), not by the FMA units.

Attainable performance at arithmetic intensity I (FLOPs per byte of memory traffic) is bounded by

$$P(I) = \min(P_{\text{peak}},\ B_\ell \cdot I), \tag{1}$$

where P_{peak} is the core's peak FLOP rate and B_ℓ the bandwidth of the memory level ℓ the data lives in [1]. We draw one bandwidth roof per level (a cache-aware roofline). Both ceilings are *measured by separate microbenchmarks* (Section 2.6), never derived from the kernels plotted against them.

10.1 Analysis

Ceilings (Table 7). The AVX2 FMA loop reaches 89.1 GFLOP/s float32 (44.6 float64), which is 31.99 FLOP/cycle at the measured 2.79 GHz. That is exactly the 2 FMA ports × 8 lanes × 2 = 32 FLOP/cycle the core can do, so the ports, not the loop, set this ceiling. The SSE mul+add loop (`autovec`'s ISA, no FMA) reaches 22.3 GFLOP/s (8.00 FLOP/cycle, expected $2 \times 4 = 8$) and the scalar loop 5.55 GFLOP/s (1.99 FLOP/cycle, expected 2). An earlier version of this report

Table 8: Arithmetic intensity (float32), counting the actual traffic per element including write-allocate, the same convention as the triad roof.

Kernel	I (FLOP/byte)	Traffic per element
SAXPY	0.167	2 FLOP/12 B: read x, read y, write y (y is read before it is written, so no write-allocate)
multiply	0.063	1 FLOP/16 B: read x, y, write z + write-allocate of z
stencil	0.417	5 FLOP/12 B: read x once (reuse), write y + write-allocate
dot product	0.25	2 FLOP/8 B: read x, y, no store
triad (roof)	0.125	2 FLOP/16 B: read b, c, write a + write-allocate

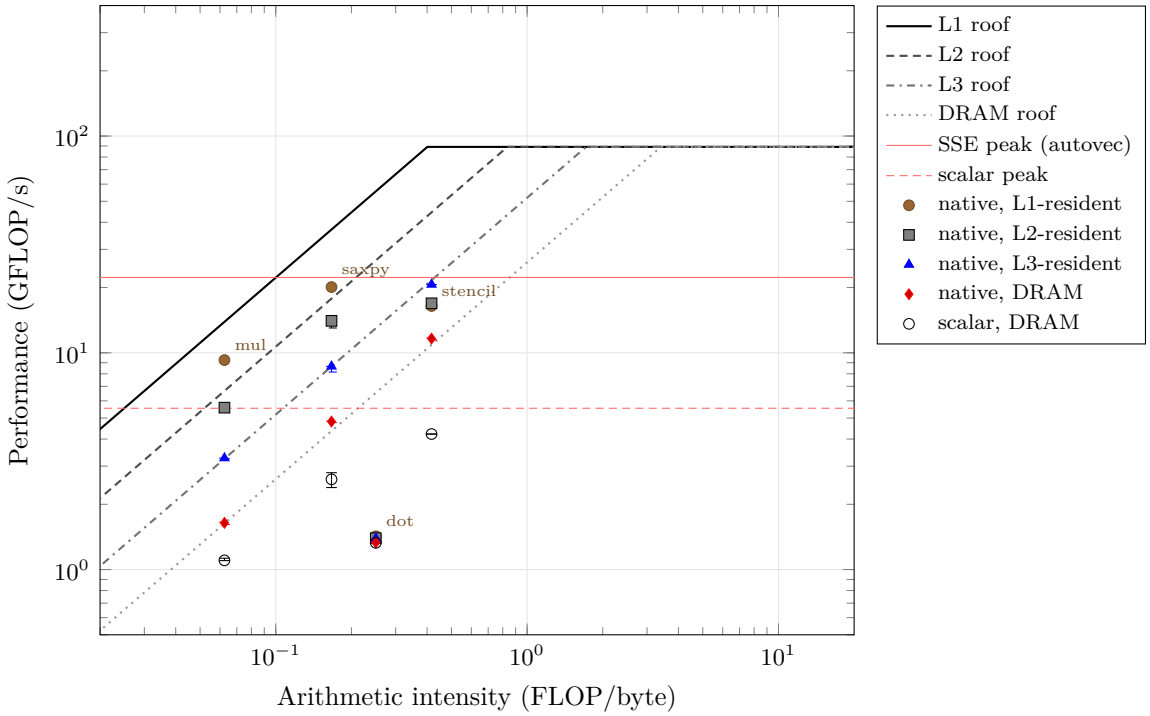


Figure 10: Cache-aware roofline for one core. Roofs: measured triad bandwidth per level, capped by the measured AVX2-FMA float32 peak; thin red lines: the measured SSE and scalar peaks that bound the `autovec` and `scalar` builds. Points: kernel GFLOP/s from Section 5 at $N = 1024$, 16 384, 524 288 and 16 777 216 (L1, L2, L3, DRAM), median with IQR error bars (mostly smaller than the markers).

estimated 147 GFLOP/s from an assumed 4.6 GHz turbo clock. The FLOP/cycle part of that estimate was right; the clock is now fixed and measured. Triad bandwidth (actual traffic) falls from 207 GiB/s in L1 to 99.4, 48.3 and 24.3 GiB/s in L2, L3 and DRAM, placing the ridge points at $I = P_{\text{peak}}/B_\ell \approx 0.40$, 0.84, 1.72 and 3.41 FLOP/B. The L1 ridge is slightly *below* the 0.5 FLOP/B the second hypothesis assumed, so the stencil ($I = 0.417$) is, in L1 only, nominally under the compute roof.

DRAM-resident kernels. Every kernel's intensity (0.06 to 0.42 FLOP/byte) is far left of the DRAM ridge, so all four are bandwidth-bound in DRAM. This is consistent with the hypothesis and with the locality sweep (Section 6). Against the DRAM roof $B_{\text{DRAM}} \cdot I$, native multiply reaches 101 % (1.64 measured vs 1.63 GFLOP/s), SAXPY 111 % (4.82 vs 4.36) and the stencil 107 %. Multiply has exactly the triad's access pattern (two reads, one write to an unread line) and lands on the roof. SAXPY and the stencil exceed it by 7 % to 11 %. Their patterns differ from triad's, and the copy stream (26.3 GiB/s actual) shows that the sustainable bandwidth depends a little on the pattern; the triad roof is a reference, not an upper bound to the byte. Because the roof now comes from a different loop, a kernel landing on it is evidence, not a tautology. The scalar DRAM points sit at 39 % to 68 % of the same roof. They are the instruction-bound loops of Section 6, which is why SIMD still pays 1.49× (multiply), 1.85× (SAXPY) and 2.76× (stencil) at these points.

Cache-resident kernels. The L3-resident native points sit at 96 % to 101 % of the L3 roof (SAXPY 100 %, multiply 101 %, stencil 96 %), so L3 bandwidth bounds them exactly. Closer to the core they fall below the roof. In L2, SAXPY reaches 79 % and multiply 84 % of the roof. In L1, native SAXPY delivers 20.1 GFLOP/s at $I = 0.167$, 54 % of the L1 roof and 23 % of the FMA peak, and the stencil reaches only 18 % of the peak. So cache-resident kernels are bounded neither by the FMA units (confirming the second hypothesis) nor quite by the measured L1 bandwidth. The limit is the loop's own load/store and instruction issue: 0.22 to 0.28 cycles per element for SAXPY versus the 0.125 that two loads and a store per cycle would allow (Section 5). The roofline thus explains the step pattern of Figure 3 from L2 outward, since each step is a move down to a lower roof. Inside L1/L2 it only gives the upper bound.

The dot product is the one kernel far *below* every roof: 1.33 GFLOP/s to 1.52 GFLOP/s at all sizes (Section 5). It is not bandwidth-bound but latency-bound by the non-reassociated reduction (Section 4). One scalar `vaddss` per element with a 4-cycle latency gives an in-core ceiling of 2 FLOP/4 cycles = 0.5 FLOP/cycle, i.e. 1.39 GFLOP/s at the measured 2.79 GHz. The measured 1.39 GFLOP/s to 1.42 GFLOP/s for every N from 1024 to 1 048 576 matches it to within 2 %; only $N \leq 512$ runs slightly above it (1.45 GFLOP/s to 1.52 GFLOP/s). Each call starts a new, independent chain, and with short chains the out-of-order core overlaps the end of one call with the next. DRAM runs slightly below it (1.33 GFLOP/s). The roofline, a throughput model, does not apply to the dot product until the dependency chain is broken. To move a streaming kernel toward the compute roof one would have to raise its arithmetic intensity (fuse passes, or block for cache so data is re-read from L1 rather than DRAM); widening the vectors alone cannot help a kernel that is already waiting on memory.

11 Negative Example

Hypothesis (from first principles, then compared with the data)

A naive benchmark—or a quickly generated one that only ever tests a small array—would conclude that SAXPY enjoys the full ≈ 6× SIMD speedup. We predict instead that the same *vectorized* kernel shows almost no speedup on a DRAM-resident array, because memory bandwidth, not

vector throughput, then sets the pace.

Case	memory-bound SAXPY: a loop that provably vectorizes (`vfmadd213ps` on `ymm`, Section 4) but whose working set far exceeds L3
Build	`native` (`-O3 -march=native`) vs `scalar`
Script	`SIMD_Advantage_Profiling/scripts/run.sh negative`

11.1 Evidence

The SAXPY loop at `kernels.c:29` is confirmed vectorized at 256 bit with FMA by both the compiler report and the disassembly in Section 4. Yet its speedup over scalar falls with size:

N	working set	native GFLOP/s	% of roof	speedup
1024	8 KiB (L1)	20.1	54	5.60×
16 384	128 KiB (L2)	13.9	78	4.19×
262 144	2 MiB (L3)	8.6	100	2.82×
4 194 304	32 MiB (DRAM)	6.1	–	2.04×
8 388 608	64 MiB (DRAM)	5.2	118	2.12×
16 777 216	128 MiB (DRAM)	4.9	112	1.87×
33 554 432	256 MiB (DRAM)	4.7	108	1.68×

"% of roof" is native GFLOP/s over $B_\ell \cdot I$ with the measured triad roof of the level the data lives in (Section 10); at 32 MiB the working set straddles L3 and DRAM. The three SAXPY sweeps (baseline, locality, this one) are separate runs. At $N = 16\,777\,216$ they give 1.85×, 1.73× and 1.87×, a spread typical of DRAM-bound throughput and larger than the within-run IQR.

11.2 Explanation

The hypothesis was half right. The $\approx 6\times$ seen in L1 indeed does not survive in DRAM, but the speedup does not fall to "almost none". It settles at 1.7–1.9×. Both parts have measured causes. (i) The loop is genuinely vectorized (disassembly), so this is not a failure to use SIMD. (ii) The kernel's arithmetic intensity (0.167 FLOP/B, Table 8) puts it deep in the bandwidth-bound region, and native SAXPY runs at 108 % to 118 % of the DRAM roof $B_{\mathrm{DRAM}} \cdot I = 4.36$ GFLOP/s. B_{DRAM} is measured by a separate triad benchmark, not by SAXPY. The LLC-miss counter gives 0.126 misses per element for native, exactly the compulsory 2/16 (Section 6). The vector loop is waiting on memory and moving no more data than it must. *No* wider vector, AVX-512 included, could make it faster. (iii) The remaining 1.7–1.9× is not a SIMD gain in the memory-bound sense: scalar SAXPY is still instruction-bound in DRAM (1.99 cycles per element at IPC 3.5, versus 1.53 with all data in L1), and the ratio of its cycles to native's (1.19) is the measured speedup. That ratio depends on the clock. With turbo enabled the same experiment gave only 1.34–1.41× (Section 12). (iv) The same kernel *does* reach 5.6× when the array fits in L1, so the hardware is capable of the speedup; in DRAM only bandwidth withholds it.

A second, subtler misleading case appears in the dot product (Section 4): its loop reports "vectorized" and the disassembly shows a 256 bit `vmulps`, so a reader who stops at the compiler report would claim success—yet the serial, non-reassociated reduction leaves the real speedup at $\approx 1\times$. Both cases carry the same lesson for AI-assisted benchmarking: "it compiled to SIMD" is necessary but not sufficient evidence that a kernel is actually faster.

12 Anomalies and Limitations

12.1 Unexpected results

- **Dot product never speeds up.** We expected a reduction to vectorize at $\approx 8\times$ like the other float32 kernels. Instead native dot stays at $\approx 1\times$ across all sizes. Hypothesis: without `-ffast-math` the compiler may not reassociate floating-point addition, so it vectorizes the multiply but keeps a serial scalar sum. Tested: the disassembly shows `vmulps` on `ymm` followed by a chain of scalar `vaddss` (Section 4), confirming the hypothesis. We chose not to enable `-ffast-math` so that all variants remain bit-reproducible against the reference.

- **Native SAXPY speedup peaks below $N = 4096$.** SAXPY reaches $6.25\times$ at $N = 512$ in the baseline and $6.9\times$ at $N = 1536$ in the locality sweep, against $5.75\times$ at $N = 4096$. We first attributed this to per-call overhead, but the tail study (Section 7) shows that fixed overhead *lowers* the native speedup at small N. The counters point elsewhere: at $N = 4096$ the two arrays fill the 32 KiB L1 exactly, and L1D misses first appear there (0.014 per element native, 0.025 scalar, versus at most 0.0002 below). The speedup peak sits at the largest N that is still fully L1-resident.

- **Stencil keeps a $2.7\times$ advantage in DRAM** while SAXPY and multiply fall to 1.7–$1.9\times$ (SAXPY) and 1.4–$1.5\times$ (multiply). Its native build is just as bandwidth-bound as theirs: it runs at 107 % of the DRAM roof (Section 10). Its scalar build, however, needs 5 FLOP per element of serial `mulss`/`addss` work and reaches only 4.0 GFLOP/s to 5.0 GFLOP/s at every size (4.2 GFLOP/s, 39 % of the roof, in DRAM). The retained ratio is again scalar's instruction cost, as for SAXPY (Section 6), just larger.

- **Cache-resident stencil is slower than L3-resident stencil.** Native stencil delivers 16.4 GFLOP/s to 17.3 GFLOP/s for $N = 1024 \ldots 16\,384$ but 19.8 GFLOP/s to 21.0 GFLOP/s for $N = 65\,536 \ldots 1\,048\,576$, and 31.4 and 25.5 GFLOP/s at the isolated sizes $N = 512$ and $32\,768$; scalar stencil stays at 4.0 GFLOP/s to 5.0 GFLOP/s. A cache effect would make the rate fall monotonically with N; this does not. Hypothesis (untested): 4K aliasing. The small `x` and `y` allocations sit at addresses whose difference makes a store to `y[i]` look like it conflicts with the load of `x[i+1]` in the low 12 address bits, which stalls the load; large allocations come from fresh pages with a different relative offset. Testing it would need a sweep over the `x`–`y` distance, which we did not run.

- **Scalar and SIMD SAXPY do not converge in DRAM.** We expected both to reach the same bandwidth plateau; native stays 1.7–$1.9\times$ ahead. Our first hypothesis was that the scalar loop keeps fewer misses in flight. The counters (Section 6) show instead that scalar SAXPY is still *instruction*-bound in DRAM (1.99 cycles per element at an IPC of 3.5, versus 1.53 in L1), while native sits at the bandwidth roof (IPC 0.6). The ratio of the two, 1.67, matches the measured speedup, and it depends on the clock: the earlier turbo-clock sweep measured only 1.3–$1.4\times$.

- **Two-state scalar SAXPY.** With L2/L3-resident data, scalar SAXPY runs at either ≈ 1.6 or ≈ 1.8 cycles per element (0.57 ns to 0.65 ns at 2.8 GHz). The state is chosen per process, independent of the code path (Section 8), and shows up as the alternating scalar points of Figure 3, and possibly also as the scalar DRAM alignment scatter. Candidate causes are physical page placement or a front-end/loop-stream-detector effect. We could not separate them without root-only tools, so cache-resident scalar baselines carry about ± 6 % of process-to-process uncertainty, which propagates into the speedups.

- **Hardware gather loses to scalar.** The explicit `vgatherdps` kernel takes 3.20 ns per element at $N = 32\,768$ versus 1.92 (native scalar loop) and 2.08 ns (scalar build), i.e. 1.5–1.7× *slower*, and it is 3 % slower in DRAM. Hypothesis: the Gather Data Sampling microcode mitigation, which this CPU runs (`/sys/devices/system/cpu/vulnerabilities/gather_data_sampling`: "Mitigation: Microcode") and which is known to make `vgatherdps` much slower, plus the eight scalar stores that AVX2 cannot vectorize. Confirming it would need a boot without the mitigation, which requires root.

12.2 Limitations

- **Limited hardware counters.** We have no root, so we count only *user-space* events of our own process through `perf_event_open` (Section 2.6): cycles, instructions, generic LLC misses and L1D read misses. Kernel-mode work, uncore/DRAM traffic counters and TLB events are not available, and the generic LLC-miss event's treatment of hardware-prefetch requests is model-specific, so we use misses per element as a relative measure between builds rather than as exact DRAM traffic.

- **Fixed clock.** All reported experiments were run on 2026-10-21 with governor performance, turbo disabled and SMT off; the cycle counter confirms 2.78 GHz to 2.79 GHz for every locality and ceilings configuration but one (Section 2). The fixed clock makes the results reproducible, but it is not how the CPU normally runs. With turbo (up to 5.2 GHz) the compute-bound numbers (peak FLOP rate, cache-resident kernels, scalar builds, the dot product) scale up by roughly the clock ratio while DRAM bandwidth does not. So the DRAM-resident speedups reported here (1.7–1.9× for SAXPY) are larger than a turbo-clocked run gives (≈ 1.4× in our earlier sweep).

- **One machine, one compiler.** All results are from a single i9-10900F with GCC gcc (GCC) 16.2.1 20260810. Clang was used only to confirm the build compiles. No AVX-512 part was available, so the 512 bit regime is out of scope.

- **THP always-on; no core isolation.** Transparent huge pages are `always`, and the pinned core is not isolated from the scheduler (no `isolcpus`). Neither is controlled for. The main session ran on an otherwise idle machine. During the alignment re-run (19:40) background work was active: a kernel worker at ≈ 12 % of CPU 0, `udev` workers, and a web browser whose threads the scheduler could place on CPU 2. We kept CPU 2 for consistency with the other experiments. The cache-resident alignment medians reproduce the first pass to within 2 %, but several configurations have upper-quartile outliers of up to 14 % (visible as one-sided error bars). The DRAM-resident results are noisier than in the first pass (Section 7).

- **Report build backend.** The build host has `bibtex` but not `biber`, so `biblatex` uses `backend=bibtex`. With that backend, the `ieee` style's citation-order sorting made every run request another BibTeX pass ("There were undefined references", although the citation resolved). This report therefore sorts its single reference by name/year, and it compiles in final mode without that warning.

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code, an AI coding agent, for coding, debugging, running the experiments and drafting this report. All sessions up to 2026-10-20 (initial implementation and first report) used the model Claude Opus 5; the 2026-10-21 rework (measured ceilings, counters, AVX2 gather, extended sweeps, the fixed-clock re-run and this revision) used Claude Opus 5.5. GCC gcc (GCC) 16.2.1 20260810 and `objdump` produced the builds and the vectorization evidence.

Who ran the measurements

The timed SIMD experiments of 2026-10-21 (`run.sh all`, and the alignment re-run) were executed by Claude Code on the user's machine, with the user's approval. Before the runs the user set the governor to `performance` and disabled turbo and SMT. Claude Code checked these settings before each run and serialized its runs with other benchmark jobs through a lock file. All measurements come from these real runs on the machine in Table 1, and every quoted number traces to their CSVs.

What they helped generate

The four kernels and their references; the build-variant Makefile; the benchmark driver and the shared timer/allocation/pinning helpers; the experiment scripts (`run.sh`), the summarizer (`summarize.c`) and the vectorization-evidence script; the roofline-ceiling microbenchmarks (`ceilings.c`), the `perf_event_open` counter helper (`perfctr.c`), the explicit AVX2 gather kernel and the post-processing scripts (`speedup_iqr.awk`, `roofline.sh`, `tables.sh`); and the first drafts of these sections.

What was independently modified, tested, or rejected

Concrete corrections, among others. (1) The generated Makefile compiled the *whole* program with each variant's flags; this was restructured so only the kernel translation unit varies, keeping the timing harness identical across builds. (2) The processed-CSV column names and one figure filter had drifted from what the plots expect—the stride figure asked for `n=16777216` while the data was measured at `n=4194304`—which would have produced a silently empty plot; the filter was corrected to the measured size. (3) *An AI-written script was simply wrong*: the `awk` in `verify_vectorization.sh` took its output path as a trailing `VAR=value` argument, which `awk` does not assign until it reaches that argument, so the `BEGIN` block saw an empty string and aborted with a null-redirection error. It was caught the first time `make report` was run and fixed by passing the path with `-v`. We also chose *not* to enable `-ffast-math`, rejecting the easy way to make the dot product vectorize, because it would break bit-reproducibility against the reference—and because the non-reassociated reduction is itself a result worth reporting (Sections 4 and 11).

Validation

Correctness: 578 reference and known-answer cases pass for every variant (Section 3). Vectorization: confirmed by compiler report and disassembly, not assumed (Section 4). Numbers: every figure traces to a committed raw/processed file (Section A); the memory-bound claims are cross-checked against bandwidth and peak FLOP rate measured by separate microbenchmarks (`src/ceilings.c`), not derived from the kernels themselves (Section 10). An earlier draft had derived the roofline bandwidth from SAXPY and then cited SAXPY landing on that line as confirmation—a circular argument that review caught and that the independent ceilings now replace. The re-run at a fixed clock also overturned two claims in the AI-drafted text, which we corrected rather than kept: that scalar and SIMD "converge" in DRAM (they stay 1.7× apart, and the counters show why), and that small-N speedups are inflated by

per-call overhead (the tail study shows the opposite). A regrade also found that the alignment sweep had silently skipped offsets 40 B to 56 B (an incomplete list in `run.sh`). The list was fixed and the sweep re-run.

14 Conclusion

On this AVX2 machine SIMD pays off handsomely only when two conditions hold at once: the data fits in cache and the kernel is not a serial reduction. Under those conditions `-march=native` gives SAXPY, elementwise multiply and the stencil roughly 3.5–5.8× over scalar at $N = 4096$ (the generic SSE build gets 2.6–3.0×), below the 8× lane count because load/store issue, not the FMA units, limits the vector loop. Once the working set spills to DRAM the vector builds run *at* the independently measured bandwidth roof, and the advantage compresses to 1.7–1.9× for SAXPY, 1.4–1.5× for multiply and 2.7–2.8× for the stencil. What remains is not extra SIMD throughput but the scalar loops' own instruction cost, which at the fixed 2.8 GHz clock still exceeds the memory time; it would shrink further at a higher clock. Data type scales the cache-resident gain through lane count (float64 halves it), and access pattern dominates everything else: any non-unit stride or a gather erases the SIMD benefit outright, and AVX2's hardware gather is even slower than the scalar loop on this mitigated CPU. The single most important lesson for writing and benchmarking vectorized code is that "the loop vectorized" is not "the loop is fast". Our memory-bound SAXPY gains nothing from more lanes, and our non-reassociated dot product, though it compiles to genuine SIMD, runs at exactly the 0.5 FLOP/cycle its dependency chain allows. Only a controlled sweep across working-set size, plus disassembly, hardware counters and a measured roofline, reveals why.

References

[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785

A Reproducibility

Code, scripts, raw data and this report live in one repository: `ACS-Code/` holds the code and data (this project in `ACS-Code/SIMD_Advantage_Profiling/`, shared helpers in `ACS-Code/common/` and `ACS-Code/tools/`), `ACS-Latex/` the reports (this one in `ACS-Latex/SIMD_Advantage_Profiling/`). Every result in this report was produced from commit `2721691f277e`. From the repository root:

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout 2721691f277e
cd ACS-Code
make tools                                  # tools/summarize
make -C SIMD_Advantage_Profiling            # scalar/autovec/native simd_bench + ceilings
make -C SIMD_Advantage_Profiling test       # 578 correctness cases
make -C SIMD_Advantage_Profiling report     # vectorization evidence
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
SIMD_Advantage_Profiling/scripts/run.sh all     # ceilings, baseline, locality, alignment,
                                                # stride, dtype, negative
scripts/sync_data.sh SIMD_Advantage_Profiling   # -> ACS-Latex/SIMD_Advantage_Profiling/data/
cd ../ACS-Latex && make simd                    # -> SIMD_Advantage_Profiling.pdf
```

Hardware counters need no root (user-space events, `perf_event_paranoid` ≤ 2); with `COUNTERS=0`, or where they cannot be opened, `run.sh` skips the counter outputs. `run.sh derive` recomputes the derived tables (roofline points and roofs, per-level bandwidth, stride summary) from the processed CSVs without measuring.

Table 9: Raw data behind each figure (directories under `ACS-Code/SIMD_Advantage_Profiling/data/raw/`).

Figure / table	Raw run directory
Figures 1 and 2	`baseline/20261021-003450`
Figures 3 to 5, Table 5	`locality/20261021-003450`
Figures 6 and 7	`alignment/20261021-194049` (`raw.csv`, `raw_tail.csv`)
Figure 8, Table 6	`stride/20261021-003450`
Figure 9	`dtype/20261021-003450`
Figure 10, Table 7	`ceilings/20261021-003450`, `baseline/20261021-003450`
Negative-case table (Section 11)	`negative/20261021-003450`
Table 4	`vectorization/20261021-001032`

Each raw directory holds `raw.csv` (one row per repetition, with full configuration metadata) and `env.txt` (a snapshot of `collect_env.sh`); the processed CSVs plotted above are produced from them by `tools/summarize` plus this project's post-processing scripts (`speedup_iqr.awk`, `roofline.sh`, `tables.sh`) and copied into this report's `data/` by `scripts/sync_data.sh`. Every number quoted in the report traces to one of these files.